

P1348R0

An Executor Property for Occupancy of Execution Agents

Published Proposal, 2018-11-25

This version:

wg21.link/P1348

Authors:

[David S. Hollman](#)

[Jared Hoberock](#)

[Lee Howes](#)

[Michael Garland](#)

[H. Carter Edwards](#)

Audience:

SG1, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Revision History

1.1 D1348R0

1.2 P1348R0

2 Summary and Motivation

3 Wording

4 Links

References

Informative References

§ 1. Revision History

§ 1.1. D1348R0

- Initial release, prepared *in situ* at the 2018-11 San Diego meeting.

§ 1.2. P1348R0

- Changed `maximum_occupancy_shape_t` to `occupancy_t` at request of SG1 at San Diego meeting.

§ 2. Summary and Motivation

We propose the addition of an optional query-only property, `occupancy_t`, with a `polymorphic_query_result_type` of `size_t`. The intention is that the result of querying this property should be used to drive the decomposition of work into parts and passed to `bulk_execute` to express the number of agents needed.

Previous discussion by authors of [P0443r9] had indicated an understanding that such a property would be added at some future time. Discussion at the 2018-11 San Diego meeting served to increase the acuteness of the need and demonstrate just how cross-cutting the concern is. At least dating back to earliest discussions of `std::reduce` parallel implementations using [P0443r9], the authors recognized this need:

```
// XXX ideally, we'd partition the input into a number of tiles
//     proportional to the "unit_shape" of the executor
//     the idea behind this property is somewhat analogous to what
//     std::thread::hardware_concurrency() reports
//     for example, a thread pool executor would probably return
//     the number of threads in the pool
//     since we don't have such a property, arbitrarily choose 16
size_t desired_num_tiles = 16;
```

(For context, see <https://gist.github.com/jaredhoberock/7888469864b45bf471e686243e8a83c7>).

Implementation reports at the 2018-11 San Diego meeting further demonstrated the ubiquity of the need for parallel algorithms is to decompose their work into tiles, and that the choice of the number of tiles is a potentially important performance concern. This number provides guidance to the parallel

algorithm calling `bulk_execute` to make an informed choice of what number of tiles they might want to use.

§ 3. Wording

Add the following property to the section enumerating the query-only properties in [\[P0443r9\]](#):

```
struct occupancy_t {
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = size_t;

    template<class Executor>
        static constexpr decltype(auto) static_query_v
            = Executor::query(occupancy_t());
};
constexpr occupancy_t occupancy;
```

Provides a nonzero estimate for the number of execution agents that should occupy associated execution contexts (if any). *[Note: For example, a thread pool executor might return the number of threads in a pool; a SIMD executor might return the number of vector lanes; a GPU executor might return the total number of hardware thread contexts; the inline executor should return 1. Unlike `std::thread::hardware_concurrency`, if this value is not well defined or not computable for a given executor type `Ex`, then `execution::can_query_v<Ex, execution::occupancy_t>` should be `false`. Provides a nonzero estimate for the number of execution agents that should occupy associated execution contexts (if any). *[Note: For example, a thread pool executor might return the number of threads in a pool; a SIMD executor might return the number of vector lanes; a GPU executor might return the total number of hardware thread contexts; the inline executor should return 2. Unlike `std::thread::hardware_concurrency`, if this value is not well defined or not computable for a given executor type `Ex`, then `execution::can_query_v<Ex, execution::occupancy_t>` should be `false`. —end note]**

§ 4. Links

- [Discussion and Issue Tracking](#)

§ References

§ Informative References

[P0443r9]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards, Gordon Brown. [A Unified Executors Proposal for C++](https://wg21.link/p0443r9). 8 October 2018. URL: <https://wg21.link/p0443r9>